

Apache Iceberg

ORC 模块架构设计文档

基于 Apache Iceberg 1.9.2 源码深度分析

[核心架构](#) · [设计模式](#) · [流程图](#) · [时序图](#) · [调用链](#)

项目	Apache Iceberg
模块	iceberg-orc
版本	1.9.2
文档类型	架构设计文档

目录

1. 模块概述与定位
2. 整体架构设计
3. 核心类一览表
4. 读取流程详解
5. 写入流程详解
6. Schema 转换与演进机制
7. 谓词下推与过滤优化
8. 指标收集系统
9. 设计模式深度剖析
10. 关键场景调用链
11. 数据层通用读写器
12. 文件 IO 适配层
13. 总结

1. 模块概述与定位

Apache Iceberg 的 ORC 模块 (iceberg-orc) 是 Iceberg 表格式与 Apache ORC 列式存储格式之间的桥梁层。它负责将 Iceberg 的逻辑 Schema 与 ORC 的物理 TypeDescription 进行双向转换，并提供统一的读写 API，使上层引擎（Spark、Flink、Trino 等）能够透明地以 ORC 格式存储和读取 Iceberg 表数据。

1.1 模块在 Iceberg 生态中的位置

iceberg-orc 位于 Iceberg 架构的 格式适配层，与 iceberg-parquet、iceberg-arrow 并列，向上对接 iceberg-core 和 iceberg-data 提供的统一抽象（如 FileAppender、CloseableIterable），向下封装 Apache ORC 库的 Reader/Writer API。其核心设计原则是引擎无关性——模块本身不依赖任何特定计算引擎。

1.2 模块统计

统计项	数值	说明
主源码文件数	27	org.apache.iceberg.orc (23) + data.orc (4)
测试文件数	13	覆盖 Schema 转换、读写、过滤等核心场景
核心入口类	ORC.java	最大文件 (844 行)，提供所有 Builder
外部依赖	Apache ORC Core	nohive 分类器，避免 Hive 依赖

1.3 核心能力

- ORC 文件读取：支持行级读取和向量化批量读取两种模式，支持 Split 级别的范围扫描
- ORC 文件写入：支持数据写入、等值删除写入、位置删除写入三种模式
- Schema 双向转换：Iceberg Schema 与 ORC TypeDescription 的完整互转，支持 Schema 演进
- 谓词下推：将 Iceberg Expression 转换为 ORC SearchArgument，实现 Stripe 级别过滤
- 指标收集：从 ORC 统计信息中提取列级 Metrics（上下界、空值计数、列大小等）
- 名称映射：支持通过 NameMapping 为无 Iceberg ID 的历史 ORC 文件添加列标识

2. 整体架构设计

2.1 分层架构图

ORC 模块采用清晰的三层架构，各层职责分明，通过接口和工厂方法解耦：

层次	核心组件	职责
API 入口层	ORC, ORCFormatModel, WriteBuilder, ReadBuilder, DataWriteBuilder, DeleteWriteBuilder	提供流式 Builder API，封装配置参数，创建读写器实例。是外部调用者的唯一入口。
核心处理层	OrcFileAppender, OrcIterable, ExpressionToSearchArgument, ORCSchemaUtil, OrcMetrics	实现文件追加/迭代的核心逻辑，包括 Schema 转换、谓词下推、指标收集等关键功能。
读写抽象层	OrcValueReader, OrcValueWriter, OrcRowReader, OrcRowWriter, OrcBatchReader, OrcValueReaders	定义值级别的读写接口，由上层引擎提供具体实现，实现引擎无关的可扩展读写框架。
Schema 访问层	OrcSchemaVisitor, OrcSchemaWithTypeVisitor, OrcToIcebergVisitor, ApplyNameMapping, IdToOrcName	基于访问者模式遍历和转换 ORC/Iceberg Schema，支持 ID 映射、名称映射等多种转换策略。

2.2 模块依赖关系

iceberg-orc 的依赖关系体现了 Iceberg 严格的模块边界设计：

依赖方向	模块	依赖内容
向上依赖	iceberg-api	Schema, Types, Expression, FileFormat 等核心抽象
向上依赖	iceberg-core	MetricsConfig, Binder, MappingUtil 等实现
向上依赖	iceberg-common	公共工具类
向下依赖	Apache ORC Core (nohive)	Reader, Writer, TypeDescription 等 ORC API
编译时依赖	Hadoop 3	Configuration, FileSystem, Path 等

3. 核心类一览表

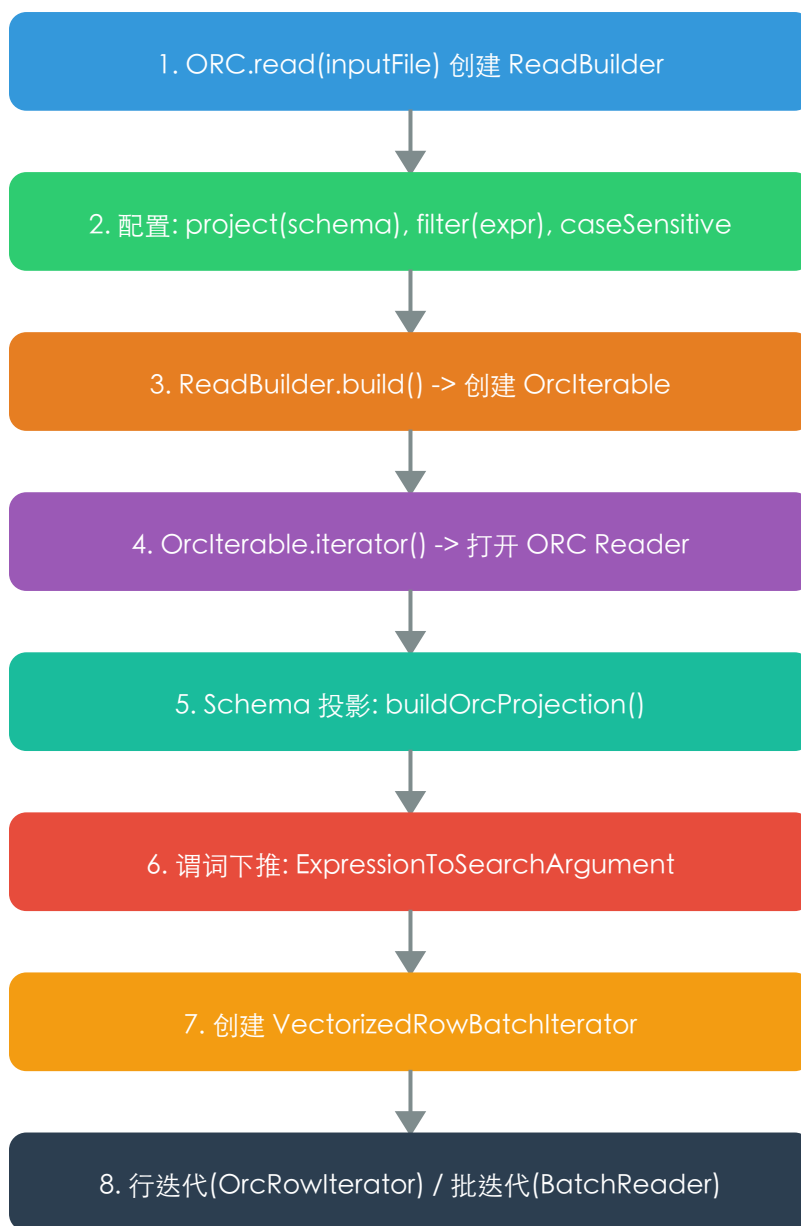
类名	包	角色	关键设计
ORC	orc	模块统一入口	静态工厂 + 嵌套 Builder 模式
ORC.WriteBuilder	orc	写入构建器	Builder 模式，配置 ORC 写参数
ORC.ReadBuilder	orc	读取构建器	Builder 模式，配置 ORC 读参数
ORC.DataWriteBuilder	orc	数据写入器构建	组合 WriteBuilder + DataWriter
ORC.DeleteWriteBuilder	orc	删除写入器构建	支持等值删除和位置删除
ORCFormatModel	orc	格式模型适配器	策略模式，适配新格式模型 API
OrcFileAppender	orc	文件追加器	VectorizedRowBatch 缓冲写入
OrcIterable	orc	文件可迭代对象	延迟求值，支持行/批两种迭代
ORCSchemaUtil	orc	Schema 工具类	双向转换 + Schema 演进
OrcMetrics	orc	指标收集器	ORC 统计信息转 Iceberg Metrics
ExpressionToSearchArgument	orc	谓词转换器	访问者模式转换表达式
OrcSchemaVisitor	orc	Schema 访问者	泛型访问者模式基类
OrcSchemaWithTypeVisitor	orc	双 Schema 访问者	同时遍历 Iceberg+ORC 类型
OrcToIcebergVisitor	orc	ORC 转 Iceberg	Schema 逆向转换
GenericOrcWriter	data.orc	通用写入器	基于 Record 的默认实现
GenericOrcReader	data.orc	通用读取器	基于 Record 的默认实现
VectorizedRowBatchIterator	orc	批次迭代适配器	ORC RecordReader 到 Iterator
FileIOFSUtil	orc	文件系统适配	FileIO 到 Hadoop FS 桥接

4. 读取流程详解

4.1 读取流程总览

ORC 文件读取是 Iceberg 查询执行中最频繁的操作之一。整个读取流程从构建 `ReadBuilder` 开始，经过 Schema 投影、谓词下推、迭代器创建等步骤，最终输出记录流。下图展示了完整的读取流程：

ORC 文件读取完整流程



4.2 读取模式对比

特性	行级读取 (OrcRowReader)	批量读取 (OrcBatchReader)
读取粒度	单行	整个 VectorizedRowBatch
接口方法	read(batch, row)	read(batch)
迭代器	OrcRowIterator (内部类)	CloseableIterator.transform
性能特点	灵活但开销较大	向量化处理，高吞吐
适用场景	通用查询、数据导出	Spark/Flink 向量化引擎
上下文传递	setBatchContext(offset)	setBatchContext(offset)

4.3 关键实现细节

Schema 投影机制：OrcIterable.iterator() 中，首先检查 ORC 文件 Schema 是否包含 Iceberg ID 属性。如果有，直接调用 ORCSchemaUtil.buildOrcProjection() 构建投影 Schema；如果没有（历史文件），则先通过 NameMapping 添加 ID，再进行投影。这一设计确保了向后兼容性。

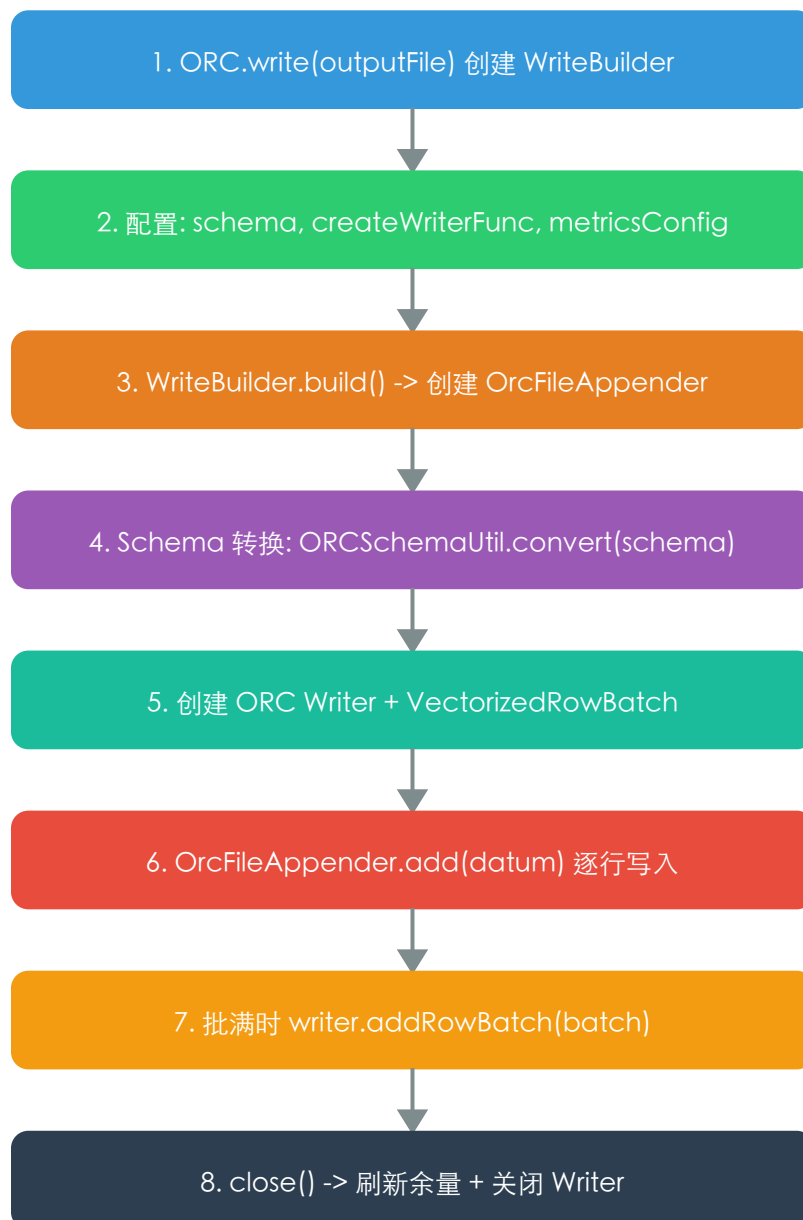
谓词下推：如果存在过滤条件，先通过 Binder.bind() 将表达式绑定到 Schema，再通过 ExpressionToSearchArgument.convert() 转换为 ORC 的 SearchArgument。SearchArgument 会被传递给 ORC 的 Reader.Options，在 Stripe 级别跳过不匹配的数据块。

VectorizedRowBatch 复用：VectorizedRowBatchIterator 内部复用同一个 batch 对象，每次调用 next() 时覆盖上一次的数据。调用方必须在下一次调用前完成数据消费。

5. 写入流程详解

5.1 写入流程总览

ORC 文件写入完整流程



5.2 三种写入模式

写入模式	入口方法	Builder 类	输出类型
数据写入	ORC.writeData()	DataWriteBuilder	DataWriter
等值删除	ORC.writeDeletes()	DeleteWriteBuilder	EqualityDeleteWriter
位置删除	ORC.writeDeletes()	DeleteWriteBuilder	PositionDeleteWriter

5.3 写入上下文 (Context)

WriteBuilder 内部的 Context 类封装了 ORC 写入的物理参数。它提供两个工厂方法——dataContext() 和 deleteContext()，分别从表属性中提取数据写入和删除写入的配置。删除写入支持独立的压缩、Stripe 大小等配置，回退到数据写入的默认值：

参数	数据写入属性	删除写入属性	默认值
Stripe 大小	write.orc.stripe-size-bytes	write.delete.orc.stripe-size-bytes	64MB
Block 大小	write.orc.block-size-bytes	write.delete.orc.block-size-bytes	256MB
批大小	write.orc.vectorbatch.size	write.delete.orc.vectorbatch.size	1024
压缩算法	write.orc.compression-codec	write.delete.orc.compression-codec	ZLIB
压缩策略	write.orc.compression-strategy	write.delete.orc.compression-strategy	SPEED
Bloom Filter 列	write.orc.bloom.filter.columns	-	""(空)
Bloom Filter FPP	write.orc.bloom.filter.fpp	-	0.05

5.4 OrcFileAppender 核心机制

缓冲写入：OrcFileAppender 维护一个 VectorizedRowBatch 作为写入缓冲。每次 add() 调用时，通过 OrcRowWriter.write() 将数据写入 batch，当 batch 满时自动刷新到 ORC Writer。close() 时会刷新最后一个不满的 batch。

文件大小估算：length() 方法在文件未关闭时返回估算值，计算公式为：已写 Stripe 大小 + (Writer 内存估算 + 当前 batch 数据量) * 0.2。这个估算值用于上层引擎判断是否需要滚动到新文件。

6. Schema 转换与演进机制

6.1 双向转换流程

Iceberg 使用基于 ID 的 Schema 演进策略，而 ORC 使用基于列名的 Schema。ORCSchemaUtil 通过在 ORC TypeDescription 的属性中嵌入 Iceberg ID 来实现两者的桥接。

Iceberg -> ORC 转换



ORC -> Iceberg 转换



6.2 类型映射表

Iceberg 类型	ORC 类型	特殊属性
BOOLEAN	BOOLEAN	-
INTEGER	INT	-
LONG	LONG	iceberg.long-type=LONG
TIME	LONG	iceberg.long-type=TIME
FLOAT	FLOAT	-
DOUBLE	DOUBLE	-
DATE	DATE	-
TIMESTAMP	TIMESTAMP / TIMESTAMP_INSTANT	iceberg.timestamp-unit=MICROS
TIMESTAMP_NANO	TIMESTAMP / TIMESTAMP_INSTANT	iceberg.timestamp-unit=NANOS
STRING	STRING	-
UUID	BINARY	iceberg.binary-type=UUID
FIXED(L)	BINARY	iceberg.binary-type=FIXED, iceberg.length=L
BINARY	BINARY	iceberg.binary-type=BINARY
DECIMAL(P,S)	DECIMAL(P,S)	-
VARIANT	STRUCT(metadata,value)	iceberg.struct-type=VARIANT

Iceberg 类型	ORC 类型	特殊属性
STRUCT	STRUCT	各字段递归转换
LIST	LIST	元素递归转换
MAP	MAP	键值递归转换

6.3 Schema 演进支持

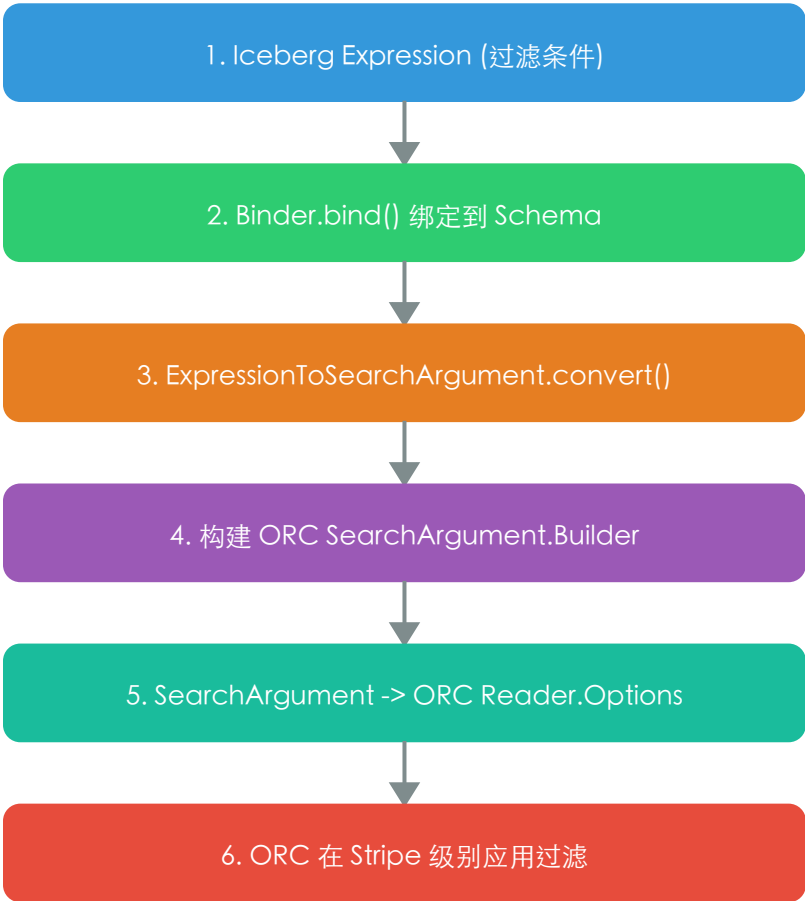
buildOrcProjection() 方法是 Schema 演进的核心实现。它将 Iceberg 的读取 Schema 映射到已有 ORC 文件的写入 Schema 上，支持以下演进操作：

- 列重命名：通过 Iceberg ID 而非列名匹配，读取 Schema 中的列名可与写入 Schema 不同
- 列重排：ORC 投影 Schema 按写入 Schema 中的列顺序排列
- 类型提升：支持 int->long, float->double, decimal(P,S)->decimal(P',S) (P'>P)
- 新增列：读取 Schema 中存在但写入 Schema 中不存在的列，尝试使用初始默认值
- 列删除：写入 Schema 中存在但读取 Schema 中不需要的列，自动跳过

7. 谓词下推与过滤优化

7.1 转换流程

Expression 到 SearchArgument 转换流程



7.2 支持的谓词操作

Iceberg 操作	ORC SearchArgument 操作	特殊处理
IS_NULL	isNull	-
NOT_NULL	startNot().isNull().end()	取反实现
IS_NAN	equals(NaN)	转等值比较
NOT_NAN	startOr().isNull().startNot().equals(NaN)	复合表达式
LT (<)	lessThan	-
LT_EQ (<=)	lessThanEquals	-

Iceberg 操作	ORC SearchArgument 操作	特殊处理
GT (>)	startNot().lessThanEquals().end()	ORC 无 GT，用取反
GT_EQ (>=)	startNot().lessThan().end()	ORC 无 GTE，用取反
EQ (=)	equals	-
NOT_EQ (!=)	startOr().isNull().startNot().equals()	保留 NULL 行
IN	in	-
NOT_IN	startOr().isNull().startNot().in()	保留 NULL 行
STARTS_WITH	literal(YES_NO_NULL)	不支持，跳过

7.3 不支持的类型

以下 Iceberg 类型无法转换为 ORC PredicateLeaf.Type，相关谓词会退化为 YES_NO_NULL（即不过滤）：BINARY, FIXED, UUID, STRUCT, MAP, LIST。此外，非 BoundReference 的 term 也无法下推。

7.4 NULL 语义差异处理

这是谓词下推中最精妙的设计之一。ORC 使用 SQL 语义——NOT_EQ 和 NOT_IN 会排除 NULL 行。但 Iceberg Expression 保留 NULL 行。ExpressionToSearchArgument 通过将 notEq(x) 转换为 OR(IS_NULL, NOT(EQ(x))) 来解决这个语义差异，确保不会错误地过滤掉 NULL 值。

8. 指标收集系统

OrcMetrics 负责从 ORC 文件的 ColumnStatistics 中提取 Iceberg 所需的列级指标信息，这些信息用于查询规划时的文件裁剪和 Stripe 跳跃。

8.1 收集的指标类型

指标名称	数据结构	说明
记录总数	long	文件总行数
列大小	Map	每列的磁盘字节数
值计数	Map	每列的值数量（含空值）
空值计数	Map	每列的空值数量
NaN 计数	Map	浮点列的 NaN 值数量
下界	Map	每列的最小值（二进制编码）
上界	Map	每列的最大值（二进制编码）

8.2 指标模式

MetricsConfig 控制每个列的指标收集粒度，支持四种模式：

- None：不收集任何指标，该列完全跳过
- Counts：仅收集值计数和空值计数
- Truncate(length)：收集完整指标，字符串类型的上下界按指定长度截断
- Full：收集完整指标，上下界保留原始值

8.3 浮点数特殊处理

ORC 的统计信息会将 NaN 包含在浮点列的上下界中，这对于文件裁剪是有害的。OrcMetrics 通过两种策略解决：如果存在 Iceberg 自行追踪的 FieldMetrics（写入时生成），直接使用其精确的上下界；否则（导入的外部文件），将 NaN 替换为正/负无穷大。

9. 设计模式深度剖析

9.1 Builder 模式 (ORC 类)

应用位置：ORC.WriteBuilder, ORC.ReadBuilder, ORC.DataWriteBuilder, ORC.DeleteWriteBuilder

设计意图：ORC

读写需要大量配置参数（Schema、压缩方式、过滤条件、批大小等），Builder 模式将复杂的对象构造过程分步化，每个方法返回 this 支持链式调用。

层次化 Builder：DataWriteBuilder 和 DeleteWriteBuilder 内部持有 WriteBuilder 实例（组合而非继承），通过委托模式复用基础写入配置，同时扩展各自特有的参数（如分区信息、删除字段 ID）。这避免了 Builder 继承链中的类型安全问题。

9.2 访问者模式 (Visitor Pattern)

应用位置：OrcSchemaVisitor, OrcSchemaWithTypeVisitor, ExpressionToSearchArgument

设计意图：ORC

Schema (TypeDescription) 是一棵类型树，需要根据不同需求对其进行多种遍历操作（ID 检查、名称映射、类型转换等）。访问者模式将遍历逻辑与处理逻辑分离，新增操作只需实现新的 Visitor。

ORC 模块中有两个层次的访问者：

访问者基类	遍历目标	具体实现	用途
OrcSchemaVisitor	ORC TypeDescription	OrcTolcebergVisitor	ORC -> Iceberg 转换
OrcSchemaVisitor	ORC TypeDescription	ApplyNameMapping	应用名称映射
OrcSchemaVisitor	ORC TypeDescription	HasIds	检查是否有 ID
OrcSchemaVisitor	ORC TypeDescription	RemoveIds	移除 ID 属性
OrcSchemaVisitor	ORC TypeDescription	EstimateOrcAvgWidthVisitor	估算行宽
OrcSchemaVisitor	ORC TypeDescription	StatsColumnsVisitor	收集统计列
OrcSchemaWithTypeVisitor	Iceberg Type + ORC	GenericOrcWriter.WriteBuilder	构建值写入器树

访问者基类	遍历目标	具体实现	用途
OrcSchemaWithTypeVisitor	Iceberg Type + ORC	GenericOrcReader.ReadBuilder	构建值读取器树
ExpressionVisitors.BoundVisitor	Iceberg Expression	ExpressionToSearchArgument	谓词下推

9.3 策略模式 (Strategy Pattern)

应用位置：OrcValueReader/OrcValueWriter 接口体系, 读写器创建函数
 设计意图：不同计算引擎（Spark、Flink）有不同的内部数据表示（InternalRow、RowData），ORC 模块通过函数式策略模式，让调用方传入 createWriterFunc/createReaderFunc 来决定具体的读写策略。

函数式策略：WriteBuilder 接受 BiFunction<Schema, TypeDescription, OrcRowWriter> 类型的策略函数；ReadBuilder 接受 Function<TypeDescription, OrcRowReader> 类型的策略函数。这比传统的策略接口更轻量，利用 Java 8 Lambda 可以直接内联策略实现。

9.4 适配器模式 (Adapter Pattern)

应用位置：FileIOFSUtil, VectorizedRowBatchIterator, ORCFormatModel
 设计意图：ORC 库依赖 Hadoop FileSystem API，而 Iceberg 使用自己的 FileIO 抽象。适配器模式在两者之间建立桥梁，使 ORC 模块可以透明地使用 Iceberg 的文件系统。

FileIOFSUtil 中的 InputFileSystem 和 OutputFileSystem 是经典的对象适配器——它们继承 Hadoop FileSystem (Target)，内部持有 Iceberg InputFile/OutputFile (Adaptee)，仅实现 ORC 实际调用的 open()/create() 方法，其他方法抛出 UnsupportedOperationException。

9.5 模板方法模式 (Template Method)

应用位置：OrcValueReader.read(), OrcValueWriter.write(), OrcValueReaders.StructReader
 设计意图：读写操作有固定的空值检查流程（模板），具体的非空值处理由子类实现。

OrcValueReader.read() 定义了读取模板：检查 isRepeating -> 检查 isNull -> 调用 nonNullRead()。OrcValueWriter.write() 定义了写入模板：检查 null -> 设置 isNull 标记 -> 调用 nonNullWrite() 或 nullWrite()。StructReader 则更进一步，在 nonNullRead() 中实现了遍历所有子字段读取器的模板逻辑，子类只需实现 create()（创建结构体实例）和 set()（设置字段值）两个抽象方法。

10. 关键场景调用链

10.1 场景一：Spark 读取 ORC 数据文件

当 Spark 引擎发起 ORC 文件读取时，完整的调用链如下（时序图形式）：

序号	调用者	方法调用	说明
1	SparkEngine	ORC.read(inputFile)	创建 ReadBuilder
2	ReadBuilder	.project(schema).filter(expr).createReaderFunc(fn)	配置参数
3	ReadBuilder	.build()	返回 OrcIterable
4	OrcIterable	.iterator()	开始迭代
5	ORC	newFileReader(file, config)	创建 ORC Reader
6	ORCSchemaUtil	hasIds(fileSchema)	检查是否有 Iceberg ID
7	ORCSchemaUtil	buildOrcProjection(schema, fileSchema)	构建投影 Schema
8	Binder	bind(schema, filter, caseSensitive)	绑定表达式
9	ExpressionToSearchArgument	convert(boundFilter, readOrcSchema)	转换为 SearchArgument
10	OrcIterable	newOrcIterator()	创建 VectorizedRowBatchIterator
11	OrcRowReader (fn)	apply(readOrcSchema)	创建引擎特定的读取器
12	OrcRowIterator	hasNext() / next()	逐行迭代返回数据

10.2 场景二：写入数据文件（含 Bloom Filter）

序号	调用者	方法调用	说明
1	DataWriter	ORC.writeData(outputFile)	创建 DataWriteBuilder
2	DataWriteBuilder	.forTable(table)	从表属性提取配置
3	DataWriteBuilder	.withSpec(spec).withPartition(partition)	设置分区信息
4	DataWriteBuilder	.build()	返回 DataWriter
5	WriteBuilder	Context.dataContext(config)	提取 ORC 物理参数
6	WriteBuilder	new OrcFileAppender(...)	创建文件追加器
7	OrcFileAppender()	ORCSchemaUtil.convert(schema)	转换为 ORC Schema
8	OrcFileAppender()	EstimateOrcAvgWidthVisitor	估算行宽

序号	调用者	方法调用	说明
9	OrcFileAppender()	ORC.newFileWriter(file, options, metadata)	创建 ORC Writer
10	DataWriter	add(record)	逐行写入
11	OrcFileAppender	valueWriter.write(datum, batch)	写入 batch
12	OrcFileAppender	writer.addRowBatch(batch)	batch 满时刷新
13	DataWriter	close()	关闭并收集 Metrics
14	OrcMetrics	fromWriter(writer, fieldMetrics, config)	提取指标

10.3 场景三：写入位置删除文件

序号	调用者	方法调用	说明
1	DeleteWriter	ORC.writeDeletes(outputFile)	创建 DeleteWriteBuilder
2	DeleteWriteBuilder	.forTable(table)	配置表属性
3	DeleteWriteBuilder	.transformPaths(fn)	设置路径转换函数
4	DeleteWriteBuilder	.buildPositionWriter()	构建位置删除写入器
5	DeleteWriteBuilder()	meta('delete-type', 'position')	设置删除类型元数据
6	DeleteWriteBuilder()	DeleteSchemaUtil.pathPosSchema()	创建 (path, pos) Schema
7	DeleteWriteBuilder()	GenericOrcWriters.positionDelete()	创建位置删除写入器
8	WriteBuilder	Context.deleteContext(config)	提取删除写入参数
9	WriteBuilder	.build() -> OrcFileAppender	创建文件追加器
10	PositionDeleteWriter	write(path, pos, row)	写入删除记录

10.4 场景四：Schema 演进读取（列重命名+类型提升）

假设原始写入 Schema 为 `struct<a: int, b: string>`，当前读取 Schema 为 `struct<a: long, c: string>`（a 类型提升为 long，b 重命名为 c）。调用链如下：

序号	调用者	方法调用	说明
1	OrcIterable	iterator()	启动读取
2	ORC	newFileReader(file, config)	打开 ORC 文件
3	ORCSchemaUtil	hasIds(fileSchema)	返回 true
4	ORCSchemaUtil	buildOrcProjection(readSchema, fileSchema)	开始 Schema 投影
5	buildOrcProjection	icebergToOrcMapping(fileSchema)	构建 ID -> OrcField 映射
6	buildOrcProjection	mapping.get(fieldId).name()	按 ID 查找原始列名
7	buildOrcProjection	getPromotedType(long, int)	检测到 int -> long 提升
8	buildOrcProjection	TypeDescription.createLong()	创建提升后的类型
9	ORC Reader	使用投影 Schema 读取	ORC 内部 Schema 演进

10.5 场景五：从无 ID 的历史 ORC 文件读取

序号	调用者	方法调用	说明
1	OrcIterable	iterator()	启动读取
2	ORCSchemaUtil	hasIds(fileSchema)	返回 false（无 Iceberg ID）
3	MappingUtil	create(schema)	从 Schema 创建 NameMapping
4	ORCSchemaUtil	applyNameMapping(fileSchema, nameMapping)	应用名称映射
5	ApplyNameMapping (Visitor)	遍历 ORC Schema，匹配字段名	为匹配的字段设置 Iceberg ID
6	ORCSchemaUtil	buildOrcProjection(schema, typeWithIds)	正常投影流程

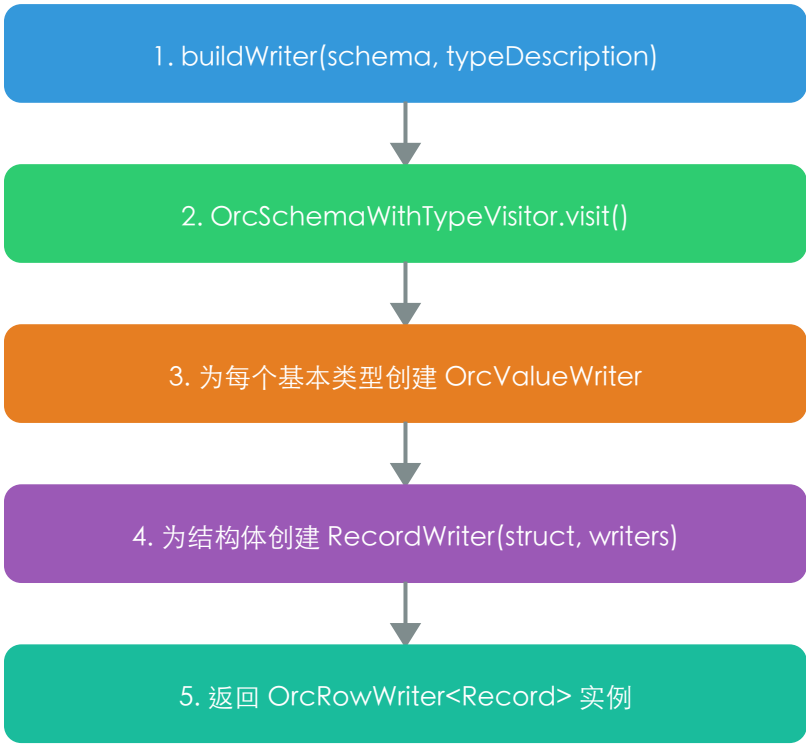
11. 数据层通用读写器

data.orc 包提供了基于 Iceberg Record 的默认 ORC 读写器实现，它们是引擎特定实现的参考模板。

11.1 GenericOrcWriter 工作机制

GenericOrcWriter 通过 OrcSchemaWithTypeVisitor 遍历 Iceberg Schema 和 ORC TypeDescription，为每个字段创建对应的 OrcValueWriter 实例，形成一棵与 Schema 结构对应的写入器树。写入时，RecordWriter（继承自 StructWriter）从 Record 中按位置取出字段值，委托给对应的 OrcValueWriter 写入 ColumnVector。

GenericOrcWriter 构建流程



11.2 GenericOrcReader 工作机制

GenericOrcReader 同样通过 OrcSchemaWithTypeVisitor 构建读取器树。它特别处理了常量字段（来自分区信息或元数据列）和特殊元数据列（ROW_POSITION, IS_DELETED）。StructReader 中的 isConstantOrMetadataField 数组标记了哪些字段不需要从 ORC 中读取。

11.3 读写器接口体系

接口/类	泛型参数	核心方法	职责
OrcValueWriter	T: 值类型	write(rowId, data, output) nonNullWrite(rowId, data, output)	单值写入接口
OrcValueReader	T: 值类型	read(vector, row) nonNullRead(vector, row)	单值读取接口
OrcRowWriter	T: 行类型	write(row, batch) writers() metrics()	整行写入接口
OrcRowReader	T: 行类型	read(batch, row) setBatchContext(offset)	行级读取接口
OrcBatchReader	T: 批类型	read(batch) setBatchContext(offset)	批量读取接口
OrcValueReaders.StructReader	T: 结构体类型	create(), set(struct, pos, value)	结构体读取模板

12. 文件 IO 适配层

ORC 库底层依赖 Hadoop FileSystem API 进行文件操作，而 Iceberg 定义了自己的 FileIO 抽象（InputFile, OutputFile）。FileIOFSUtil 和 ORC 工具方法共同构成了适配层。

12.1 适配策略

InputFile 类型	适配方式	FileSystem 来源
HadoopInputFile	直接获取 Hadoop FileSystem	file.getFileSystem()
其他 InputFile	包装为 InputFileSystem	FileIOFSUtil.InputFileSystem

OutputFile 类型	适配方式	FileSystem 来源
HadoopOutputFile	直接获取 Hadoop FileSystem	file.getFileSystem()
其他 OutputFile	包装为 OutputFileSystem	FileIOFSUtil.OutputFileSystem

12.2 NullFileSystem 设计

InputFileSystem 和 OutputFileSystem 都继承自 NullFileSystem——一个所有方法都抛出 UnsupportedOperationException 的基类。这种设计确保只有 ORC 实际调用的方法被实现（open 和 create），避免了不必要的接口实现负担，同时如果 ORC 库未来调用了新方法会立即报错而非产生默默的错误行为。这是空对象模式（Null Object Pattern）的变体，体现了防御性编程原则。

13. 总结

13.1 设计亮点

- 引擎无关架构：通过函数式策略模式（createWriterFunc/createReaderFunc），ORC 模块完全不依赖任何计算引擎的数据类型，Spark、Flink、Trino 等引擎只需提供各自的读写器工厂函数即可接入
- 基于 ID 的 Schema 演进：通过在 ORC TypeDescription 属性中嵌入 Iceberg field ID，实现了与列名无关的 Schema 匹配，支持列重命名、列重排等操作
- 精确的 NULL 语义：ExpressionToSearchArgument 通过将 NOT_EQ/NOT_IN 转换为 OR(IS_NULL, NOT(...)) 的形式，完美解决了 Iceberg 和 ORC 之间的 NULL 处理语义差异
- 访问者模式的深度运用：模块中有 8+ 个不同的 Visitor 实现，充分展示了如何通过访问者模式实现对复杂类型树的多种操作，避免了大量的 switch-case 重复代码
- 向后兼容的名称映射：对于缺少 Iceberg ID 的历史 ORC 文件，自动通过 NameMapping 基于字段名建立映射，保证了存量数据的可读性
- 分离的数据/删除写入上下文：数据文件和删除文件支持独立的物理配置（压缩、Stripe 大小等），允许针对删除文件的 IO 模式进行独立优化

13.2 设计模式总览

设计模式	应用位置	解决的问题
Builder	ORC.WriteBuilder / ReadBuilder 等	复杂对象的分步构造
Visitor	OrcSchemaVisitor 体系	类型树的多种遍历操作
Strategy	createWriterFunc / createReaderFunc	引擎无关的读写扩展
Adapter	FileIOFSUtil	FileIO 到 Hadoop FS 的桥梁
Template Method	OrcValueReader / Writer	读写操作的空值处理模板
Factory Method	ORC.write() / read() 等静态方法	创建 Builder 实例
Null Object	NullFileSystem	安全的默认行为
Composition	DataWriteBuilder 持有 WriteBuilder	替代继承的灵活复用

本文档基于 Apache Iceberg 1.9.2 源码分析生成，深入剖析了 iceberg-orc 模块的架构设计、核心流程、设计模式和关键调用链。